

engenharia de software.

O **Modelo Cascata (Waterfall)** baseia-se em uma abordagem estritamente sequencial, o que traz previsibilidade para cenários estáveis, mas gera alta rigidez perante imprevistos.

Desafio 1 — Identificando as fases

Objetivo principal de cada fase:

- **Requisitos:** Levantar, documentar e validar todas as necessidades e funcionalidades do sistema junto ao cliente.
- **Análise:** Modelar o sistema e entender o domínio do problema, definindo diagramas, regras de negócio e restrições.
- **Projeto (Design):** Desenhar a arquitetura técnica, definindo banco de dados, interfaces e tecnologias a utilizar.
- **Implementação:** Traduzir o projeto em código real (programação e desenvolvimento do software).
- **Testes:** Validar o software codificado para encontrar falhas (bugs) e garantir que ele atende aos requisitos iniciais.
- **Manutenção:** Corrigir problemas pós-lançamento, atualizar o sistema e adaptá-lo ao ambiente de produção.

Por que uma fase só começa após o término da anterior?

O modelo é puramente linear e baseado em "entregáveis" (documentos ou artefatos). A saída de uma fase serve como a entrada obrigatória para a próxima. Não há como projetar sem a análise pronta, nem codificar sem o desenho da arquitetura concluído.

O que acontece se uma fase for mal executada?

Ocorre o **efeito cascata de erros**. Se os requisitos forem mal levantados, a análise será falha, o design será incorreto e o código será construído errado. O erro acumula e se torna exponencialmente mais caro para corrigir nas fases finais (Testes/Manutenção).

Desafio 2 — Simulando o fluxo do projeto

-

- **Permite voltar facilmente?** Não. O modelo cascata tradicional é rígido. Voltar atrás quebra a premissa do fluxo linear e exige reabrir documentações e aprovações que já estavam congeladas.
 - **Impactos em prazo e custo:** O impacto é massivo. Como a equipe já está codificando, descobrir um erro de requisito exige refazer a análise, redesenhar a arquitetura e reescrever o código. Isso gera atrasos severos no cronograma e estouro do orçamento limitado.
 - **Reflexão sobre projetos rígidos:** Projetos rígidos sofrem com a "ilusão de certeza". Eles assumem que humanos não erram na comunicação inicial. Quando o erro aparece tardiamente, o modelo pune a equipe com retrabalho destrutivo.
 -
-

Desafio 3 — Vantagens do modelo cascata

- - **Situações em que funciona bem:** Projetos pequenos, simples, com tecnologias dominadas pela equipe e onde os requisitos são totalmente claros, fixos e imutáveis desde o primeiro dia (como no caso proposto do sistema de pedidos).
 - **Benefício de requisitos estáveis:** Se os requisitos não mudam, a equipe ganha máxima eficiência. Não há perda de tempo mudando o código no meio do caminho, permitindo um desenvolvimento direto e sem desvios.
 - **Relação com planejamento e documentação:** Como tudo é definido no início, é muito fácil estipular cronogramas exatos, datas de entrega e custos. Além disso, cada fase gera uma documentação rica e detalhada, o que facilita a entrada de novos membros na equipe.
 -
-

Desafio 4 — Limitações do modelo cascata

-
- **Dificuldade em lidar com mudanças:** O modelo trata a mudança como uma falha de processo, e não como uma constante natural. Ele não possui mecanismos nativos para abraçar alterações no meio do fluxo.

- **Como gera retrabalho:** Se o cliente pede uma mudança na fase de implementação, partes inteiras do software que já foram codificadas e testadas precisam ser descartadas e refeitas do zero, jogando fora horas de trabalho remunerado.
- **Riscos de insatisfação do cliente:** O cliente só vê o software funcionando no final do processo (fase de testes). Se houver um desalinhamento de expectativas, ele passará meses esperando para, no final, receber algo que não atende ao que ele realmente precisava, gerando frustração extrema.
-

Desafio 5 — Avaliação crítica

-
- **Indicado para projetos longos e complexos?** Não. Em projetos longos, o mercado muda, o negócio do cliente evolui e novas necessidades surgem. Usar cascata nesses cenários é extremamente arriscado e costuma levar ao fracasso do projeto.
- **Importância de escolher o modelo adequado:** O ciclo de vida escolhido dita o sucesso operacional. Escolher o modelo errado (como usar Cascata em um ambiente incerto, ou Ágil em um contrato rigidamente engessado de escopo fechado) gera atrito, prejuízo e atrasos.
- **Por que não existe um modelo "perfeito"?** Porque os projetos possuem naturezas diferentes. Projetos de software para sistemas médicos ou de aviação exigem o rigor e a documentação do Cascata. Aplicativos de celular inovadores ou startups exigem a velocidade, feedback e adaptação dos Métodos Ágeis (Scrum/Kanban). O contexto sempre dita a regra.
-